

Natív Programozás ZH

SZTE Szoftverfejlesztés Tanszék
2025. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
 - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot g++ fordítóval és a
-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1
paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
 - Futási hiba 6: Memória- vagy időkorlát túllépés.
 - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
 - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlinde克斯, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen .cpp forrásfájlt lehet feltölteni (zip fájlt nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.

Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy "3 alma", akkor az elvárt kimenet idézőjelek nélkül az 3 alma, de a szóköz szükséges!
 - A tesztesetekben nem lesz ékezetes szöveg kiírása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
 - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- A ZH során használható segédanyag elérhető bíróban.
 - <https://biro.inf.u-szeged.hu/kozos/native/>

Dinamikus memória követése

Ebben a feladatban lehetőség van egy speciális konstrukció használatára ami segít az allokált memória követésére. Amennyiben a `new` után egy `TRACK_INFO`-t írunk a `biro` rendszere ki tudja írni, hogy hol került allokálásra az adott memória terület ami nem lett felszabadítva (bizonyos tesztek esetén). Ezt az információt a `teszteset.stdout` kiterjesztésű fájljába írja bele.

Példa használat: `new TRACK_INFO Sword(...)`

Példa kimenet az `stdout` fájlban:

```
[ FAIL ] Memory error occured. Here is the map of the allocated memory addresses:
---| FAIL | 0xd7ee40 is not freed. filename: feladat.cpp:123 in function: <function>
---| FAIL | 0xd7eeb0 is not freed. filename: feladat.cpp:123 in function: <function>
---| OK   | 0xd7ef20 is nicely freed.
---| OK   | 0xd7ef50 is nicely freed. filename: feladat.cpp:133 in function: <function>
```

Figyelem! Amennyiben nem `biro-n` kerül fordításra a program, ilyen memória allokáció követés nem történik.

Általános követelmények

- Minden osztály másolható legyen (másoló konstruktor és értékadó operátor ha szükséges).
- Minden osztály megfelelően szabadítsa fel az általa birtokolt erőforrásokat.

Feladatok

Könyv osztály

A `Könyv` osztály egy könyvet reprezentál.

- A könyvnek van címe, szerzője (mindkettő szöveg) és kiadási éve (pozitív egész szám).
- Legyen egy háromparaméteres konstruktora, ami könyv adatait kapja a fenti sorrendben, és ezekkel inicializálni az objektumot.
- Emellett a könyvet lehessen lájkolni is.
 - Kezdetben egyetlen lájkja sincs a könyvnek.
 - A „klasszikus” `pre++` operátorral lehessen lájkolni a könyvet, és ilyenkor eggyel nő a lájkok száma.
- Az `std::string` konverziós operátor adja vissza a könyv adatait az alábbi formában de **NE** legyen sortörés a végén.
 - A formátum: `"<szerzo>: <cim> (<ev>), lajk: <lajkok szama>"`
 - * Példa: `"Iro: Könyv (2022), lajk: 2"`

GyerekKönyv osztály

A gyerekkönyv olyan könyv, amelyik kifejezetten gyerekeknek készült.

- A **GyerekKönyv** osztály a **Könyv** osztályból származik, és bővíti azt korhatárral (egész szám), ami megmondja, hogy hány éves kortól ajánlott a könyv.
- A konstruktorában a könyv 3 tulajdonsága után meg lehessen adni a korhatárt is.
- Definiálja felül az `ősosztály std::string` konverziós operátorát, hogy az a következő formátumban adja vissza a könyv adatait (és **NE** legyen sortörés a végén):
 - "`<szerzo>: <cim> (<ev>, korhatar: <korhatar>), lajk: <lajkok szama>`"
 - * Példa: "Valaki: B&B (2022, korhatar: 6), lajk: 5"

TudomanyosKönyv osztály

A tudományos könyv olyan könyv, ami valamelyik speciális tudományterületről szól.

- A **TudomanyosKönyv** osztály a **Könyv** osztályból származik, és bővíti azt a tudományterülettel (szöveg).
- A konstruktorában a könyv 3 tulajdonsága mellett a tudományterületet is meg lehessen adni negyedik paraméterben.
- Definiálja felül az `ősosztály std::string` konverziós operátorát, és a következő alakban adja vissza a adatokat (**NE** legyen sortörés a végén):
 - "`<szerzo>: <cim> (<kiadasi ev>, <tudomanyterulet>), lajk: <lajkok szama>`"
 - * Példa: "BS: C++ felso fokon (2021, IT), lajk: 6"
- Tudományos könyvek esetén a lájk „kettőt ér”, ezért az osztály definiálja felül a `pre ++` operátorát úgy, hogy az kettővel növelje a lájkok számát.

Ekönyv osztály

Az ekönyv „nem egy valódi könyv”, hanem egy könyvet lehet rátölteni, és azt lehet olvasni.

- A **Könyv** osztályból származzon.
- Legyen default konstruktora, ilyenkor nem tartalmaz könyvet.
- Legyen egy olyan konstruktora, ami egy **Könyv** pontert vár, és ez lesz az a könyv, amit az ekönyvön lehet olvasni.
 - Az ekönyv felel a paraméterben kapott könyv felszabadításáért.
 - Ekönyvre biztosan nem töltünk ekönyvet.
- Az ekönyv `std::string`-gé konvertálása a rátöltött könyv konvertálását jelenti.
 - Ha az ekönyvön nincs könyv, akkor "Az ekönyv ures" szöveget kell visszaadni (és **NEM** kell sortörés a végére).

- Ha ekönyvben lájkolunk a könyvet (`operator++`), akkor az háromszoros szorzót jelent ahhoz képest, mintha könyvet közvetlenül lájkoltuk volna.
 - Például ha egy gyerekkönyvnek van 12 lájkja, és az ekönyvben lájkoljuk egyszer, akkor 15 lájkja lesz.
 - Ha nincs benne könyv, akkor ne lájkoljon semmit se.

Könyvespolc osztály

A könyvespolc könyveket tárol és egymás után tesszük fel a könyveket a polcra, soha sem szúrunk be a korábban feltettek elé.

- Legyen default konstruktora az osztálynak.
- A `<<` operátorral lehessen könyvet hozzáadni a könyvespolchoz.
 - A paramétere egy `Könyv` pointer, és az osztály felel a paraméterben kapott objektum megfelelő felszabadításáért.
 - Az operátor legyen láncba fűzhető.
- Legyen egy `getKönyvek` metódusa, ami egy `string`-ben visszaadja a polcon lévő könyveket azok eredeti sorrendjében.
 - A könyvek `kiir` metódusát kell használni.
 - Sortöréssel kell elválasztani a könyveket (az utolsó után is legyen sortörés).
- A polcról az utolsó könyvet a `!` operátorral lehessen levenni.
 - `Könyv` pointer legyen a visszatérési típusa.
 - A polc nem felel többé a visszaadott könyv felszabadításáért.
 - Ha üres a polc, akkor `nullptr-t` kell visszaadni.
- Legyen egy indexer operátora, ami úgy adja vissza a könyvet, hogy azt ki lehessen cserélni a polcon egy másik könyvre.
 - Nem megfelelő index esetén `out_of_range` kivételt kell dobni.
- Valósítsd meg a `||` operátort, aminek mindkét operandusa `Könyvespolc`, és a következő módon működik:
 - Ha a jobb oldali polcon több könyv van, mint a baloldalin, akkor a bal oldali polcról az összes könyvet **átrakja** a jobb oldalra.
 - Különben a jobb oldalról rakja át a könyveket a bal oldalra.
 - Azt a polcot adja vissza, amelyiken a könyvek vannak átrakás után.

Algoritmusok

Az algoritmusokat a `"#if defined(MY_LIBRARY) || !defined(TEST_BIRO)"` makrófeltétellel védett területen kell megvalósítani, különben a bíróba feltöltve fordítási hibát okoz a feladat többi részének.

findLibrary függvény

- Készíts egy `findLibrary` függvényt, ami egy `Library`-eket tartalmazó `std::vector`-t és egy `string`-et kap paraméterben, és a `test::find_if` segítségével megkeresi a vektorban az első olyan könyvtárat, aminek az `id`-je megegyezik azzal, ami a paraméterben érkezett. Adjon vissza igazat a függvény, ha talált ilyen `id`-jú könyvtárat. Adjon vissza hamisat, ha nem található ilyen könyvtár.

biggestLibrary függvény

- Valósítsd meg az `biggestLibrary` nevű függvényt, ami egy `Library`-ket tartalmazó `std::vector`-t kap paraméterben, és a `test::max_element` alkalmazásával megkeresi a legnagyobb könyvtárat, azaz azt a könyvtárat, amiben a legtöbb letárolt könyv van. Adj vissza a könyvek darabszámát ebben a könyvtárban. Üres vektor esetén nullát adjon vissza. A `test::max_element` az `std::max_element(3)`-at használja, azaz azt a verziót, amiben az első két paraméter egy-egy iterátor, a harmadik paraméter pedig egy úgynevezett comparator (lambda). A `test::max_element`-et is pontosan így kell felparaméterezni. **Tipp:** a comparator két könyvtárat hasonlít össze és igazat ad vissza, ha a bal oldali könyvtár "kisebb".